

Trilogue アーキテクチャ決定記録 (ADR)

目的: 本プロジェクトで「なぜその作りにしたのか」を、後から（他のメンバー・数年後の自分・卒論執筆時に）追跡できるように記録する。Phase 1 の立ち上げから現在までを時系列で遡って整理したもの。

各項目は **背景** → **決定** → **理由** → **結果と副作用** → **検討した代替案** の形式で記述する。「状態」は Accepted（採用中） / Superseded（後の決定で置き換え） / Proposed（未実施）で示す。

一覧

#	決定	時期	状態
ADR-001	Next.js / TypeScript を土台にする	2026-04	Accepted
ADR-002	LLMプロバイダを Gemini から OpenAI へ変更	2026-04	Accepted
ADR-003	LabVIEW をクライアントにし HTTP 直結とする (DLL化しない)	2026-06	Accepted
ADR-004	LabVIEW 専用エンドポイントを Web 用とは別に設ける	2026-06	Accepted
ADR-005	HTTP ステータスは常に 200 とし、成否は <code>ok</code> フラグで返す	2026-06	Accepted
ADR-006	既製の LLM ツールを使わず、接続層を自作する	2026-06	Accepted
ADR-007	サーバーはステートレスとし、会話履歴はクライアントが持つ	2026-06	Accepted
ADR-008	エージェントを ID で切り替える構造を先行して入れる	2026-06	Accepted
ADR-009	APIキーはサーバー内のみで保持し、クライアントへ渡さない	2026-06	Accepted
ADR-010	上限値とタイムアウトを共通の定数として持つ	2026-06	Accepted
ADR-011	LLM 呼び出しをプロバイダ抽象の背後に置く	2026-06	Accepted
ADR-012	外部通信をモックしたユニットテストを置く	2026-06	Accepted
ADR-013	Docker を後追いで導入する	2026-07	Accepted
ADR-014	マルチエージェント化を A/B/C の3段階に分ける	2026-06	Accepted
ADR-015	発言順の決定と実行を分離する	2026-07	Accepted
ADR-016	複数AIの応答を、配列と連結文字列の両方で返す	2026-07	Accepted
ADR-017	各AIに直前までの発言を読ませる	2026-07	Accepted

#	決定	時期	状態
ADR-018	一部のAIが失敗しても会話全体は返す（部分成功）	2026-07	Accepted
ADR-019	未知の <code>agentId</code> は既定値に丸めずエラーにする	2026-07	Accepted
ADR-020	対外資料ではペルソナを抽象的に描く	2026-06	Accepted
ADR-021	ブランチを main / develop / 作業ブランチの3層にする	2026-08	Accepted

ADR-001: Next.js / TypeScript を土台にする

時期: 2026-04（Phase 1）／状態: Accepted

背景

チャットUIとAI呼び出しの両方が必要で、限られた期間で動くものを作る必要があった。

決定

Next.js（App Router / Route Handlers）と TypeScript を採用し、画面とAPIを同一プロジェクトに置く。

理由

- 画面とサーバーAPIを1つのリポジトリ・1つの言語で扱えるため、往復の型を共有できる
- APIキーをサーバー側に隠す構成が、追加の仕組みなしに実現できる
- TypeScript の型定義を、フロント・サーバー・（後の）LabVIEW契約の共通仕様書として使える

結果と副作用

- 型定義がそのままAPI仕様の役割を果たし、後の LabVIEW 連携時に契約を明文化しやすかった
- 一方で「Next.js に依存している」ため、**完全な自作とは言えない点**を対外的に明示する必要が生じた（→ ADR-006）

検討した代替案

- Python（FastAPI 等）**：LLM周辺のライブラリは豊富だが、画面を別に用意する必要がある
- LabVIEW から直接 LLM を叩く**：APIキーがクライアントに露出し、認証やエラー処理をLV上で書くことになる（→ ADR-003 で明確に否定）

ADR-002: LLMプロバイダを Gemini から OpenAI へ変更

時期: 2026-04 (Phase 1 途中) / 状態: Accepted (当初の Gemini 採用を Superseded)

背景

Phase 1 当初は Gemini API (`gemini-1.5-flash`) で実装していたが、 **APIクォータ (利用制限)** により開発中の試行が滞った。

決定

OpenAI API (`gpt-4o-mini`) へ切り替える。モデル名は環境変数で変更可能にする。

理由

- 開発中に試行回数の制限で止まると、検証サイクルが回らない
- モデル選択自体は研究の本質ではなく、**接続インフラの構築が主眼**であるため、安定して回せる方を選ぶ判断でよい

結果と副作用

- 検証が滞らなくなった
- 「特定ベンダーに固定されるのでは」という懸念が残ったため、後に抽象化した (→ ADR-011)
- 未使用となった Gemini の依存パッケージは削除し、ドキュメントの記述も OpenAI に統一した

検討した代替案

- クォータ緩和を待つ / 有料枠を申請する**: 時間がかかり、Phase 1 の進行が止まる
- ローカルLLM**: 計算リソースが不足しており現時点では非現実的 (将来の検討事項)

ADR-003: LabVIEW をクライアントにし HTTP 直結とする (DLL化しない)

時期: 2026-06 (Phase 2-A) / 状態: Accepted

背景

画面デザインの変更が今後多数発生することが見込まれ、Web UI 側で都度対応するのは非効率だった。また、計測・制御系との連携や複数AIへの拡張も視野に入っていた。LabVIEW 側で C を書ける、DLL を取り込めるという選択肢も検討対象に挙がった。

決定

LabVIEW を **HTTPクライアント**とし、LabVIEW 標準の HTTP Client VIs だけで サーバーと JSON をやり取りする。**DLL化は行わない**。

理由

- 画面の変更を LabVIEW 側だけで完結でき、サーバーに手を入れずに済む
- LabVIEW 標準機能のみで完結するため、ビルド環境やバージョン依存の問題が発生しない
- 通信が JSON over HTTP なので、LabVIEW 以外のクライアント（他アプリ）からも同じAPIを使う
- 「楽で汎用化できる方向」という方針に合致する

結果と副作用

- LabVIEW 2016（研究室の環境）でも問題なく動作し、日本語も文字化けしなかった
- 別PC間（Windows の LabVIEW ↔ Mac のサーバー）でも同一LANで動作した
- サーバーが常に必要な構成になった（＝サーバーの可搬性が課題化 → ADR-013）

検討した代替案

- **DLL化してLabVIEWに内包**: 技術的には可能だが、ビルド・配布・デバッグの手間が大きく、開発効率を損なう。得られる利点（サーバー不要）に見合わない判断
- **LabVIEW から直接 LLM を叩く**: APIキーの露出、TLS・認証・リトライ処理をLV上で実装する必要があり、現実的でない

ADR-004: LabVIEW 専用エンドポイントを Web 用とは別に設ける

時期: 2026-06（Phase 2-A）／状態: Accepted

背景

既存の Web UI 用API は、入れ子構造の `messages[]`（`id`・`createdAt` 等を含む）を要求していた。これを LabVIEW で組み立てるのは手間が大きい。

決定

Web用APIとは別に、**平坦な JSON** を受け取る LabVIEW 専用エンドポイントを新設する。

理由

- LabVIEW の `Flatten To JSON` / `Unflatten From JSON` は、**クラスタのラベル名がそのまま JSONのキーになる**ため、平坦な構造ほど組み立て・解析が容易
- Web UI の都合（メッセージID・タイムスタンプ等）を LabVIEW に押し付けたくない
- 双方の要求が将来分岐しても、互いに影響しない

結果と副作用

- LabVIEW 側は「クラスタを1つ作って配線する」だけで送受信できるようになった
- エンドポイントが増えるため、**内部のLLM呼び出し部分は共通部品として共有**する必要が生じた（そのように整理済み）

検討した代替案

- **Web用APIを LabVIEW でも使う**: LabVIEW 側の実装が複雑になり、変更のたびに影響する
- **Web用APIを平坦化して統一**: Web UI 側が持つべき情報（ID等）を失う

ADR-005: HTTP ステータスは常に 200 とし、成否は `ok` フラグで返す

時期: 2026-06（Phase 2-A）／状態: Accepted

背景

LabVIEW の HTTP Client VIs では、HTTPステータスコードによる分岐処理が煩雑になる。一方でエラー内容はユーザーに見せる必要がある。

決定

LabVIEW 向けエンドポイントは、**エラー時も HTTP 200 を返す**。成否は本文中の `ok`（真偽値）で表し、理由は `error`（文字列）に入れる。

理由

- LabVIEW 側は「`ok` を Case Structure に繋ぐ」だけで分岐でき、実装が最小になる
- 成功時も失敗時も**同じ構造の JSON** が返るため、解析部分を1つで済ませられる
- エラー文言をそのまま画面に出せる

結果と副作用

- LabVIEW 側の VI が大幅に簡素になった

- 一般的な REST の作法からは外れるため、**この方針は LabVIEW 向けエンドポイントに限定し**、Web 用APIは通常どおりステータスコードで表現している

検討した代替案

- **標準的なステータスコードを返す**: 作法としては正しいが、LabVIEW 側の分岐が増える。クライアントが LabVIEW に限定されている現状では利点が薄い

ADR-006: 既製の LLM ツールを使わず、接続層を自作する

時期: 2026-06 / 状態: Accepted

背景

Dify / Flowise / LiteLLM / NextChat / OpenWebUI といった既製の LLM 製品があり、これらを使えばマルチエージェント的な機能も短期間で構築できる。一方で、それらを使いながら「自作」と表現するのは研究発表として不適切である、との指摘があった。

決定

既製の LLM 製品は**使用しない**。接続・振り分け・会話制御は独自に実装する。そのうえで、**土台として使っている OSS (Next.js / Docker 等) は明示する**。

理由

- 本研究の主題が「**LLM と LabVIEW を JSON で接続するインフラの構築**」であり、そこを既製品に置き換えると研究の中身が無くなる
- 既製品はブラックボックスが大きく、LabVIEW 固有の要求（平坦JSON・`ok` フラグ等）に合わせ込むのがかえって難しい
- 研究としての誠実さを担保するため、自作範囲と借用範囲の線引きを明文化する

結果と副作用

- 対外的な表現を「**標準的なOSSフレームワークとDockerの上に、LabVIEW-LLM接続インフラを自作した**」に統一した。「完全自作」とは言わない
- 実装工数は増えるが、要求に対して過不足のない構成になった

検討した代替案

- **既製品を採用し、その上で研究する**: 構築は速いが、研究の中核が外部実装になる
- **既製品を参考にしつつ自作**: 実質的にこの方針（設計思想は参考にしている）

ADR-007: サーバーはステートレスとし、会話履歴はクライアントが持つ

時期: 2026-06 (Phase 2-A) / 状態: Accepted (将来見直しの可能性あり)

背景

文脈のある対話には会話履歴が必要だが、サーバー側にセッション管理やDBを導入すると構成が一気に複雑になる。

決定

サーバーは履歴を保持しない。文脈が必要な場合は、**クライアントが履歴を毎回リクエストに含めて送る**方式とする。

理由

- セッション管理・DB・認証といった要素を導入せずに済み、構成が単純に保てる
- サーバーを再起動しても状態が壊れない
- 複数のクライアント (LabVIEW / Web / 将来の他アプリ) が同じAPIを共有しやすい

結果と副作用

- サーバー側の実装が簡潔になり、Docker化も容易だった
- リクエストサイズが履歴の分だけ増えるため、**件数の上限**が必要になった (→ ADR-010)
- **患者ごとの会話ログの蓄積・時系列解析**は別途仕組みが必要 (今後の課題として明記)

検討した代替案

- **サーバー側にセッションを持つ**: 履歴送信は不要になるが、状態管理と永続化が必要
- **DBを導入する**: 将来的には必要になる見込みだが、現時点では時期尚早

ADR-008: エージェントを ID で切り替える構造を先行して入れる

時期: 2026-06 (Phase 2-A) / 状態: Accepted

背景

Phase 2-A の目標は「LabVIEWから投げて返る」ことだけだったが、マルチエージェント化が次の目標であることは分かっていた。

決定

単体AIしか使わない段階でも、`agentId` で人格（システム指示）とモデルを切り替える構造を 先に入れておく。

理由

- 後から差し込むと、リクエスト形式の変更＝LabVIEW側の作り直しになる
- 構造を先に入れておけば、LabVIEW 側は「`agentId` を変えるだけ」でAIごとの差を試せる
- 実装コストが小さいうちに入れる方が安い

結果と副作用

- 段階B（複数AIの一括応答）を実装する際、**リクエスト形式の破壊的変更が不要**だった
- エージェントの追加は定義を1つ足すだけで済む構造になった

検討した代替案

- **必要になってから入れる**: 実装は先送りできるが、後の変更範囲が広がる

ADR-009: APIキーはサーバー内のみで保持し、クライアントへ渡さない

時期: 2026-06 / 状態: Accepted

背景

研究室で共有されるコードであり、複数のPCで動かす前提がある。APIキーは有料枠に紐づくため、漏洩の影響が大きい。

決定

APIキーは**サーバーの環境変数のみ**で保持する。LabVIEW にも Web フロントにも渡さない。実キーはリポジトリに含めず、**プレースホルダのみのテンプレート**を配布する。

理由

- クライアントに渡すと、VIファイルやスクリーンショット経由で流出しうる
- リポジトリに含めると、公開時・共有時に取り返しがつかない

結果と副作用

- テンプレートファイルを用意し、各自が自分のキーを設定する運用とした
- 実キーを含むファイルはバージョン管理から除外している

- Docker 化の際も、**イメージに焼き込まず実行時に渡す**構成とした（→ ADR-013）

検討した代替案

- **クライアントから渡す**: LabVIEW 側でキーを扱うことになり、露出範囲が広がる
-

ADR-010: 上限値とタイムアウトを共通の定数として持つ

時期: 2026-06 / 状態: Accepted

背景

ステートレス方式（ADR-007）では履歴が無制限に伸びうる。また、LLM の応答が返らない場合に処理が滞留する懸念があった。

決定

会話履歴の件数、1 発話の最大文字数、LLM 呼び出しのタイムアウトを **共通の定数として一箇所に定義し**、全エンドポイントで共有する。

理由

- エンドポイントごとに値がずれると、挙動の違いが原因の分かりにくい不具合になる
- 上限は「利用料」「応答時間」「安定性」に直結するため、明示的に管理したい

結果と副作用

- 段階B（複数AI）の実装時に、**発言数の上限**を同じ場所に追加するだけで済んだ
- 上限を超えた場合は、LLM を呼ぶ前に拒否するため無駄な課金が発生しない

検討した代替案

- **各エンドポイントで個別に定義**: 重複と不整合の温床になる
-

ADR-011: LLM 呼び出しをプロバイダ抽象の背後に置く

時期: 2026-06 / 状態: Accepted

背景

ADR-002 でプロバイダを一度切り替えており、将来も変わりうる（ローカルLLM、別ベンダー、医療データを外部に出せない場面など）。

決定

LLM 呼び出しを**インターフェース越し**に行い、具体的な実装（現在は OpenAI）を その背後に置く。エラーも共通の分類（レート制限・上流エラー・内部エラー）に正規化する。

理由

- プロバイダ差し替え時の変更を1箇所に閉じ込められる
- 呼び出し側（各エンドポイント）が、ベンダー固有のエラー形式を知らずに済む
- テスト時にモックへ差し替えやすい（→ ADR-012）

結果と副作用

- 段階B の複数AI実行も、同じ呼び出し口をそのまま使えた
- 現時点で実装は1つ（OpenAI）のみで、抽象が「使われていない」状態ではあるが、過去に一度切り替えた実績があるため妥当と判断している

検討した代替案

- **直接SDKを呼ぶ**: 記述量は減るが、切り替え時に呼び出し箇所すべてに影響する

ADR-012: 外部通信をモックしたユニットテストを置く

時期: 2026-06 / 状態: Accepted

背景

API の入力検証や分岐を手動確認していたが、変更のたびに実際のLLMを呼ぶのは 時間・費用の両面で無駄が大きい。

決定

Vitest を導入し、**LLM 呼び出しをモックしたユニットテスト**を置く。

理由

- 検証ロジック（必須項目・長さ制限・上限超過）は LLM に到達する前に動くため、外部通信なしで確認できる

- 課金も発生せず、実行が速いため頻繁に回せる

結果と副作用

- 段階B の実装時、**発言順・文脈の受け渡し・部分成功・上限超過**といった 中核の挙動をテストで固定でき、手戻りを防げた
- 既存エンドポイントの内部整理（共通化）を行った際も、壊れていないことを即座に確認できた

検討した代替案

- **実際のLLMを呼ぶ結合テスト**: 現実に近いが、遅く・不安定で・課金される。最終確認としては別途手動で実施している

ADR-013: Docker を後追いで導入する

時期: 2026-07 / 状態: Accepted (当初の「Docker不使用」を Superseded)

背景

当初は Docker を使わず、Node.js を直接動かす構成だった（開発の即時性を優先）。その後、「**そのサーバーが別のPCで動くかどうかの検証**」が課題として挙げられた。実際、研究室の Windows 機では Git・Node.js・パッケージマネージャのいずれも 未インストールで、環境構築の段階で行き詰まった。

決定

Docker（および Compose）を導入し、**1コマンドで起動できる**構成を追加する。ただし **Docker 無しでも動く構成は維持**する。

理由

- 環境構築で詰まる要素（Node.js のバージョン、Git の有無）を丸ごと排除できる
- 研究室の他のPCへ移す、数年後に別の学生が引き継ぐ、といった場面で再現性が担保される
- 二重に経路を用意しておけば、開発時は直接実行、配布時は Docker と使い分けられる

結果と副作用

- **ZIPでダウンロードした状態から、Docker のみで起動し応答まで到達**することを実証した（Node.js・Git は一切インストールしていない）
- APIキーはイメージに焼き込まず、実行時に外部から渡す構成とした（ADR-009 の維持）
- 初回のイメージ作成に数分かかる点は許容している

検討した代替案

- **Node.js の導入手順を丁寧に書く**: 実際に試したが、環境差で詰まる箇所が多く断念
- **最初から Docker で開発する**: 開発中の再起動が遅くなり、初期の試行錯誤には不向きだった。
「後から追加する」判断は結果的に妥当だったと考えている

ADR-014: マルチエージェント化を A/B/C の3段階に分ける

時期: 2026-06 / 状態: Accepted

背景

最終的な目標は「司会AIが会話を仕切り、複数の専門AIが偏りなく議論する」構成だが、一度に作ると、設計が固まらないまま大きな実装を抱え込む恐れがあった。

決定

次の3段階に分けて進める。

段階	内容
A	LabVIEW 主導で、AIを1体ずつ順番に呼ぶ（最小の「AI同士の会話」）
B	サーバー側オーケストレーション：1リクエストで複数AIの応答をまとめて返す
C	ファシリテーター（司会）AI+発話順のポリシー化

理由

- 各段階で「動くもの」が手に入り、そのつど方向性を確認できる
- 段階Aは既存の仕組みだけで実現でき、**追加実装なしに構想を体験できる**
- 段階Cは臨場的な知見が必要な部分を含むため、先に土台を固めておきたい

結果と副作用

- 段階B までを実装済み。各段階が報告の単位としても機能している
- 段階A は「やればできる」ことの確認にとどめ、実装の主軸は段階B に置いた

検討した代替案

- **一気に段階Cまで作る**: 会話設計（誰にいつ話させるか）が未確定のまま作ることで、作り直しのリスクが高い

ADR-015: 発言順の決定と実行を分離する

時期: 2026-07（段階B）／状態: Accepted

背景

段階Bでは「順番に呼ぶ」だけでよいが、段階Cでは「**司会AIが次の発話者を決める**」ことになる。ここを一体で書くと、段階Cで大幅な作り直しが発生する。

決定

「誰が何番目に話すかを決める処理」と「実際にAIを順に呼ぶ処理」を分離し、前者を差し替え可能な形にする。

理由

- ・ 段階C では、決定側だけを AI の判断に置き換えれば済む
- ・ 発言順の決め方（均等巡回／文脈依存の指名／発言量の調整）を、実行側に影響を与えずに試せる
- ・ テストの観点でも、順序決定の検証と実行の検証を分けられる

結果と副作用

- ・ 段階C への拡張点が構造として明示された（報告時の説明もしやすい）
- ・ 現時点では決定側の実装が1つだけで、やや過剰に見える構造ではある

検討した代替案

- ・ 一体で実装し、段階Cで書き直す: 短期的には簡潔だが、拡張時の影響範囲が読めない

ADR-016: 複数AIの応答を、配列と連結文字列の両方で返す

時期: 2026-07（段階B）／状態: Accepted

背景

複数AIの発言を返す場合、素直には「発言の配列」になる。しかし LabVIEW で配列（クラスタの配列）を解析するのは、単一文字列に比べて手数が多い。一方で「AIごとに色を変える」等の要求には配列が必要になる。

決定

両方返す。発言ごとの詳細を持つ配列に加えて、全発言を1本に連結した文字列を同じ応答に含める。

理由

- LabVIEW 側は「成否フラグ」と「連結文字列」の2つだけ見れば会話を表示できる
- 応答に余分なキーがあっても、LabVIEW の JSON 解析は**指定した項目だけ拾って残りは無視する**ため、配列を無視しても壊れない
- 将来 AIごとの個別表示が必要になったら、同じ応答の配列を解析すればよい（サーバー変更不要）

結果と副作用

- 既存の単体AI用 VI を複製し、**変更4箇所**で複数AI版が完成した
- 応答サイズは重複分だけ増えるが、テキストのため実用上は問題にならない

検討した代替案

- **配列のみ返す**: LabVIEW 側の実装が増える。「LV側の実装を増やさない」方針に反する
- **連結文字列のみ返す**: 個別表示や失敗した発言の識別ができなくなる

ADR-017: 各AIに直前までの発言を読ませる

時期: 2026-07（段階B）／状態: Accepted

背景

複数AIに同じ入力を渡して並列に呼ぶ方が、実装は単純で高速（並行実行できる）。しかし目指しているのは「**会話**」であり、単なる複数回答の列挙ではない。

決定

AIを**逐次**呼び出し、各AIには**直前までの発言を文脈として渡す**。発言者が区別できる形で記録し、「複数のAIが順番に発言する場である」「既出の意見を繰り返さない」ことを各AIに伝える。

理由

- 並列だと、各AIが同じ内容を別の言い回しで返すだけになりやすい
- 互いの発言を踏まえることで、補足・反論・視点の追加といった**多角性**が生まれる
- 「偏らない会話空間」という最終目標に向けて、この性質が前提になる

結果と副作用

- 実測で、2体目が「ただし」と受けて慎重な補足に回り、3体目がさらに別の角度から広げる挙動を確認した

- 応答時間がAIの数に比例して伸びる（3体で約6秒）。並列化すれば短縮できるが、会話が成立しなくなるため受け入れている
- LabVIEW 側の通信タイムアウト設定に注意が必要になった（既定値では不足しうる）

検討した代替案

- 並列に呼んで結果を並べる: 高速だが会話にならない
- 1体目だけ独立、2体目以降は文脈あり: 中途半端で、利点が乏しい

ADR-018: 一部のAIが失敗しても会話全体は返す（部分成功）

時期: 2026-07（段階B）／状態: Accepted

背景

複数のAIを順に呼ぶため、途中の1体だけがレート制限やタイムアウトで失敗する場合がある。このとき全体をエラーにするか、取れた分を返すかの判断が必要だった。

決定

失敗した発言は**その発言単体の失敗として記録**し、残りのAIの呼び出しは続行する。取得できた発言はすべて返す。全体の成否フラグは「全発言が成功したときのみ真」とする。

理由

- 3体中2体の発言が取れているのに全部捨てるのは、利用者にとって損失が大きい
- どの発言がなぜ失敗したかが個別に分ければ、原因の切り分けができる
- 全体フラグを厳格（全成功のみ真）にしておけば、異常を見落とさない

結果と副作用

- LabVIEW 側では、失敗した発言も会話の流れの中にエラー表示として現れる
- 「一部成功」という状態が存在するため、成否の意味を仕様として明記する必要性が生じた

検討した代替案

- 1体でも失敗したら全体を失敗にする: 単純だが、実用上もったいない
 - 失敗を無視して成功分だけ返す: 異常に気づけなくなる
-

ADR-019: 未知の `agentId` は既定値に丸めずエラーにする

時期: 2026-07（段階B）／状態: Accepted

背景

単体AI用のエンドポイントでは、未知の `agentId` が来たら既定のエージェントで応答する（寛容な）挙動にしていた。複数AI指定でも同じにするか検討した。

決定

複数AI指定の場合は、未知の ID を **明示的なエラー** として返す（既定値に丸めない）。

理由

- 複数指定でスペルミスがあると、同じAIが2回発言する等の不可解な結果になり、原因の特定が難しい
- 単体指定なら「意図と違うが動く」で済むが、複数では取り違えの影響が大きい
- エラー文言に指定可能なIDを含めれば、その場で自己解決できる

結果と副作用

- 指定ミスが即座に分かるようになった
- 同一エンドポイント群の中で挙動が揃っていない（片方は寛容、片方は厳格）ため、仕様書に理由とともに明記している

検討した代替案

- **既定値に丸める**: 一貫性はあるが、デバッグが困難になる
- **単体側も厳格にする**: 既存のLabVIEW VI の挙動を変えることになるため見送り

ADR-020: 対外資料ではペルソナを抽象的に描く

時期: 2026-06／状態: Accepted

背景

マルチエージェントの構成図を描く際、各AIに具体的な役割名を当てはめる案があった。一方で、役割の設計自体はまだ確定しておらず、臨床的な検討が必要な段階である。

決定

構成図・対外資料では、専門AIを「差し替え可能なペルソナ」として抽象的に描き、具体的な役割名は置かない。

理由

- 未確定の役割を図に固定すると、それが決定事項として一人歩きする
- 本質は「**役割を動的に切り替えられる仕組み**」であり、個々の役割名ではない
- 役割の設計は臨床の知見が必要な領域であり、こちらで先に決めるべきではない

結果と副作用

- 図が汎用的になり、役割が変わっても資料を作り直さずに済む
- 実装側も、特定の役割を前提にしない構造を保っている

検討した代替案

- **具体的な役割名を置く**: 分かりやすいが、未確定事項を確定として示すことになる

ADR-021: ブランチを main / develop / 作業ブランチの3層にする

時期: 2026-08 / 状態: Accepted

背景

動作している状態を壊さずに新機能を試したい、という要求があった。また、`main` が Phase 1 の古い状態のまま止まっており、リポジトリを共有された人が古い構成を手にする問題があった。

決定

- **作業ブランチ**: 機能ごとに作成し、そこで実装・検証する
- **develop**: 検証が済んだ作業ブランチを統合する
- **main**: 共有・配布用。動作確認済みの状態のみを反映する

理由

- 実験が失敗しても、動いている状態はそのまま残る
- 研究室メンバーが見る `main` を常に「動く最新」に保てる
- 作業ブランチを残しておけば、経緯の追跡ができる

結果と副作用

- 段階Bの実装を、既存ブランチに影響を与えずに進められた
- `main` が Phase 1 から現行構成へ更新され、共有時の齟齬が解消された
- ブランチ操作の手数は増えるが、単独開発でも取り違えの防止に有効

検討した代替案

- `main` に直接コミット: 手軽だが、壊れた状態が共有されるリスクがある

今後の決定が必要な事項（Proposed）

以下は未決定であり、判断の材料が揃った時点で ADR として追記する。

論点	現状	判断に必要なもの
ファシリテーターAIの指名方式	未実装（構造のみ用意）	どの場面でどの役割に話させるか、臨床側の知見
会話ログの蓄積方式	サーバーは保持しない（ADR-007）	患者ごとの時系列解析の要件
通信の暗号化（HTTPS化）	LAN内のHTTPのみ	実データを扱う時期・運用場所
ローカルLLMの採用	外部APIを利用	計算リソースの確保、個人情報の取り扱い方針
センサーデータの統合方法	未着手	取得データの形式・粒度（他メンバーの進捗と連動）
同時発言させるAIの数	上限を設定済み、実用は3体程度	実際の対話における体感速度の評価